SYSTEMBESCHREIBUNG & ARCHITEKTURDOKUMENTATION
BIRDS AI SOUND CLASSIFIER

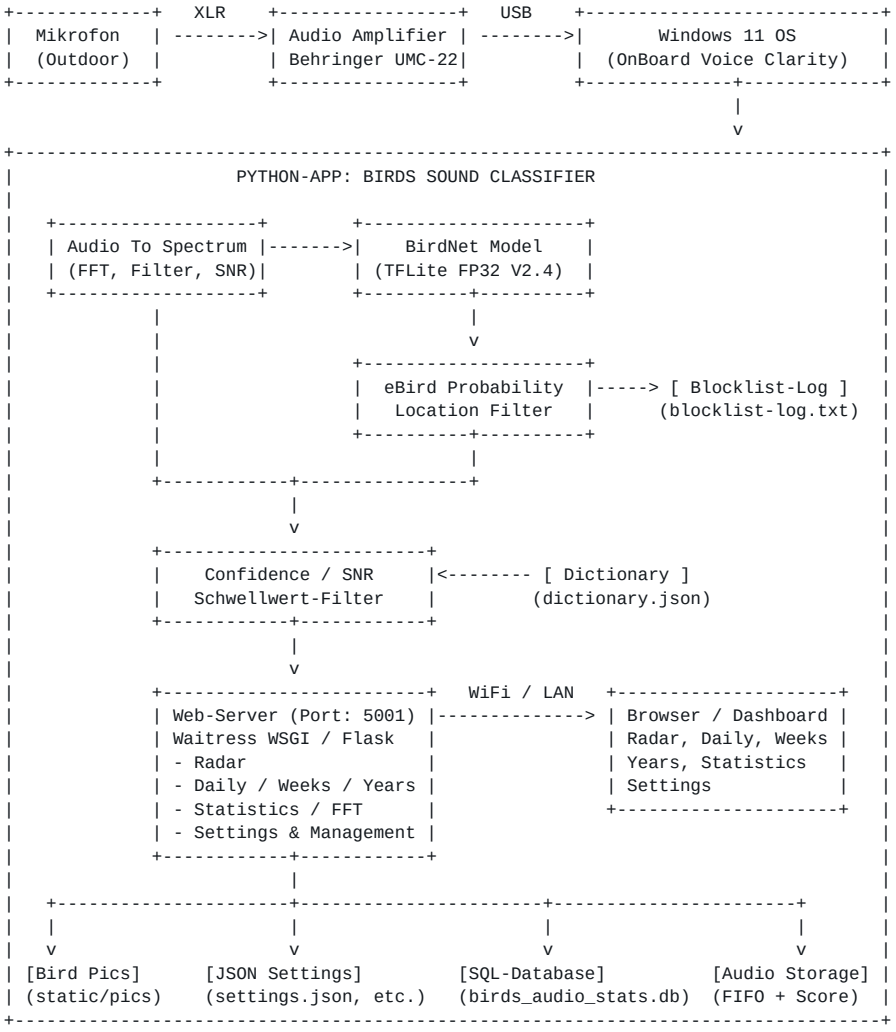
Projekt-Repository: https://github.com/mobifu1/birds_ai_sound_classifier
Entwicklungsansatz: Gemini Agent-based Vibecoding (Google DeepMind / Antigravity)
Betriebssystem: Microsoft Windows 11 OS
Stand / Version: V1.3.8-RC2 (Oktober 2026)

1. EXECUTIVE SUMMARY & SYSTEMZWECK

Der "Birds AI Sound Classifier" ist ein hochgradig spezialisiertes, autonomes Bioakustik-System zur kontinuierlichen Echtzeit-Erfassung, Identifikation, Filterung, Archivierung und statistischen Auswertung von Vogelstimmen.

Das System kombiniert moderne physikalische Signalerfassung (XLR-Mikrofonie, Vorverstaerkung), digitale Vorfilterung und Fast-Fourier-Transformation (FFT) mit einem Convolutional Neural Network (CNN) auf Basis des BirdNET-Modells unter TensorFlow / TFLite. Ueber ein zweistufiges Filtersystem (akustische Konfidenz + geografischer eBird-Wahrscheinlichkeitsabgleich) und fortgeschrittene Schwellwert-Logiken (SNR, Occurrence Threshold, Auto Season Lowering) liefert die Anwendung praezise ornithologische und bioakustische Echtzeitdaten. Die Ergebnisse werden ueber einen integrierten Webserver (Port 5001) in einem interaktiven Dashboard (akustisches 2D-Radar, Tages-, Wochen-, Jahresansichten, FFT-Periodizitaetsanalysen, Territorien- und Nachtaktivitaets-Kacheln) im lokalen Netzwerk visualisiert.

2. SYSTEMARCHITEKTUR & SIGNALFLUSS ("HOW IT WORKS")



3. HARDWARE & SIGNALKETTE (BIOAKUSTIK-FRONTEND)

3.1 Mikrofon (Mic):

- Spezielles Aussenmikrofon / Richtmikrofon zur Erfassung feiner bioakustischer Frequenzen im Bereich von ca. 200 Hz bis ueber 15.000 Hz.
- Symmetrische Signalfuehrung ueber hochwertige XLR-Verkabelung zur Eliminierung von elektromagnetischen Einstreuungen ueber laengere Leitungswege.

3.2 Audio-Vorverstaerker & Interface (Amplifier):

- Behringer U-Phoria UMC-22 Audio Amplifier & Interface.
- Ausgestattet mit einem rauscharmen MIDAS-Design Mikrofonvorverstaerker inklusive +48V Phantomspeisung fuer Kondensatormikrofone.
- Praezise Analog-Digital-Wandlung (ADC) und Weiterleitung per USB-Audio-Stream.

3.3 Betriebssystem (OS: Microsoft Windows 11):

- Robuste 64-Bit Host-Plattform fuer 24/7-Dauerbetrieb.
- OnBoard Voice Clarity / Windows Audio Stream: Native Abtastung und Vorfilterung ueber PyAudio / WASAPI.

- Prozess-Ueberwachung und Auto-Recovery ueber Windows-Subsysteme.

4. DIGITALE SIGNALVERARBEITUNG, FFT & AUDIO-VORFILTERUNG

4.1 Audio-Pipeline-Parameter:

- Sampling-Rate: 48.000 Hz (BirdNET Standard-Abtastrate)
- Kanal: 1 Kanal (Mono), 16-Bit PCM
- Blockgroesse (Chunk): 1024 Samples
- Analyse-Fenster: 3,0 Sekunden (144.000 Samples)
- Ueberlappung (Overlapping): 1,5 Sekunden Versatz fuer lueckenlose Ueberwachung.

4.2 Vorfilterung:

- High-Pass Filter: Herausfiltern von tiefen Stoergeraeuschen (z. B. Wind, Rumpeln, Nahverkehr, Infraschall) mit einstellbarer Grenzfrequenz (Standard: 500 Hz).
- Low-Pass Filter: Abschneiden hochfrequenter Artefakte (z. B. Grillen, Zirpen, Elektronikrauschen) mit konfigurierbarer Grenzfrequenz (Standard: 7.000 Hz).
- Rauschunterdrueckung (Noise Reduction): Zuschaltbare dynamische Rauschminderung via 'noisereduce'-Bibliothek (Profile: Low, Medium, High).
- Signal-to-Noise Ratio (SNR): Kontinuierliche Berechnung des Signal-Rausch-Abstands in dB zur Bewertung der akustischen Signalqualitaet.

4.3 Spektrale Visualisierung & FFT-Analysen:

- Audio To Spectrum: Schnelle Fourier-Transformation (FFT) zur Erzeugung von Echtzeit-Wasserfall-Diagrammen (Waterfall Display im Dashboard).
- Forensische Phasenstabilitaet: Visualisierung der Phasendifferenzen (-180 Grad bis +180 Grad). Ermoeglicht die Differenzierung zwischen organischen Rufen (wabernde, dynamische Phasenstrukturen) und technischen Stoersignalen (starre, homogene Phasenlinien).
- Biologische FFT-Periodizitaetsanalyse: Langzeit-FFT zur Erkennung biologischer Rhythmen der Vogelaktivitaet:
* 30-Tage-Periode: Lunarer Zyklus (Vogelzug, Nachtaktivitaet bei Vollmond).
* 24-Stunden-Periode: Zirkadianer Tag-/Nacht-Rhythmus.
* 12-Stunden-Periode: Morgen- und Abendchor (akustische Ausbreitungsoptima).
* 8h / 6h / 5h-Perioden: Stoffwechselzyklen, Siesta-Pausen, Fuetterungsfluege und Brutwechsel-Intervalle.

5. KUeNSTLICHE INTELLIGENZ & BIRDNET-MODELL (TENSORFLOW / TFLITE)

5.1 KI-Modell & Runtime:

- BirdNET_GLOBAL_6K_V2.4_Model_FP32.tflite (Full-Precision 32-Bit Modell).
- Trainiert auf ueber 6.522 weltweite Vogelarten (Klassenliste: BirdNET Labels).
- Ausfuehrung ueber TensorFlow / TFLite-Runtime, optimiert fuer latenzarme Inferenz auf CPU- oder GPU-Architekturen.

5.2 Zweistufiges Filterkonzept:

- Stufe 1: Akustische Konfidenz (Raw Confidence)
Das neuronale Netz analysiert das 3-Sekunden-Spektrogramm und gibt fuer jede Art einen Konfidenzwert (0 % bis 100 %) aus.

- Stufe 2: Geografischer Wahrscheinlichkeitsfilter (eBird Location Filter)
Abgleich der Kandidatenliste mit der globalen eBird-Beobachtungsdatenbank auf Basis der konfigurierten GPS-Koordinaten (Breite/Laenge) und der Kalenderwoche (KW). Verhindert Verwechslungen heimischer Voegel mit exotischen Doppelgaengern.

5.3 Schwellwerte & Plausibilisierung:

- Occurrence Threshold (OccT):
Historische eBird-Statistikhuerde (Standard 0.03 = 3 % der Checklisten). Reduziert die 6.000 globalen Arten auf die ca. 50-150 saisonal realistischen Kandidaten am Standort.
- Auto Season Lowering (ASL):
Dynamische, kalenderwochenspezifische Absenkung des Schwellenwerts (gesteuert ueber 'auto_season_lowering.json'), um Zugvoegel und spaete Nachzuegler im Herbst/Fruehjahr nicht faelschlich abzuweisen.
- Force & Blocked Overrides (Woerterbuch):
* 'Force': Ignoriert den Geo-Filter fuer eine Art (z. B. fuer seltene Gaeste).
* 'Blocked': Unterdrueckt Arten gezielt (Ausschluss von Fehlalarmen).
- Blocklist-Logging ('blocklist-log.txt'):
Transparente Dokumentation aller Arten, die akustisch erkannt, aber durch den Geo-Filter oder Sperrlisten verworfen wurden.

6. INTELLIGENTE ERFASSUNGSLOGIK & SPAM-SCHUTZ

6.1 1-Fenster-Verzoegerung mit Peak-Erkennung:

Erkennungen werden um 1,5 Sekunden (ein halbes Fenster) gepuffert. Das System vergleicht die Konfidenz des ersten Fensters mit dem direkt folgenden, ueberlappenden Fenster (in dem der Ruf oft zentriert liegt) und archiviert nur das Signal mit dem maximalen Konfidenzwert.

6.2 Intelligenter Streak-Spamschutz:

Ruft derselbe Vogel ununterbrochen ueber mehrere Fenster hinweg, wird er als "aktiver Streak" markiert und nicht mehrfach in die Datenbank geschrieben. Erst nach einer Pause / Stille wird der Streak zurueckgesetzt.

6.3 Multitasking-Erkennung:

Rufen mehrere Vogelarten gleichzeitig im selben 3-Sekunden-Fenster, werden alle Arten parallel bewertet, auf Schwellwerte geprueft, getrennt gespeichert und auf dem Radar visualisiert.

7. WEBSERVER, DASHBOARD & BENUTZEROBERFLAeCHE (PORT 5001)

7.1 Backend-Architektur:

- Python Flask Webframework in Kombination mit Waitress Production WSGI Server.
- ZeroConf / mDNS-Broadcasting ('http://bird-ai-sound-classifier.local:5001').
- Multiprocessing-Architektur: Trennung von Audioaufnahme, KI-Inferenz und Webserver-Bedienung zur Vermeidung von Audio-Dropouts.

7.2 Dashboard-Module & Views:

- Akustisches 2D-Radar (-Radar):
Zweidimensionale Visualisierung aller aktuellen Erkennungen. Die Distanz zum Zentrum repraesentiert die Signalstaerke (SNR), der Polarwinkel bzw. die Positionierung spiegelt die zeitliche Historie wider.
- Tagesansicht (-Daily):
Chronologische Timeline des aktuellen Tages mit Daemmerungs- und Sonnenstands-Berechnung (Astral / SunCalc.js).
- Wochen- & Jahresansichten (-Weeks / -Years):
Aggregierte Balkendiagramme je Kalenderwoche, Gegenueberstellung mit eBird-Wahrscheinlichkeitskurven und ASL-Grenzen, CSV-Exportfunktion.
- Spezies-Profile (-Statistics / species.html):
Detaillierte Einzelartenauswertungen inklusive 24h-Zeitradar, Frequenzspektrum, Phasenstabilitaet und Amplitudenverlauf.
- Territorien & Schnittmengen (-Territory / -Intersection):
Erfassung akustischer Revieraktivitaeten und Co-Vorkommen ueberlappender Arten.
- Nachtaktivitaet (-Prediction / Kachel 7):
Spezialueberwachung naechtlicher Flug- und Zugrufe von Schwarm- und Zugvoegeln.
- Einstellungen & Administration (-Settings):
Kindersicherung / Passwortschutz, Live-Audio-Pegelanzeige, Live-Spektrogramm, Audio-Filtereinstellungen, Woerterbuch-Duplettencheck und DB-Backups.

8. PERSISTENZ, DATENHALTUNG & SCHNITTSTELLEN

8.1 SQL-Datenbank ('birds_audio_stats.db'):

- SQLite-Relationaldatenbank mit Indizierung auf Zeitstempel, Artname, SNR, Konfidenz und Dateipfade.
- Automatische Backup- und Konsistenzpruefungsroutinen.

8.2 Konfigurationsdateien (JSON):

- 'settings.json': Globale Laufzeitkonfiguration (Thresholds, GPS, Radar, Filter).
- 'dictionary.json': Vogelarten-Woerterbuch mit Uebersetzungen, individuellem OccT, individueller Konfidenz, Phaenologiestatus und Aufenthaltsmonaten.
- 'auto_season_lowering.json': Wochenspezifische Absenkungsparameter.
- 'pushover.json': Zugangsdaten und Filter fuer Smartphone-Push-Benachrichtigungen.
- 'birdweather.json': API-Token fuer das globale BirdWeather-Netzwerk.

8.3 Duales Audio-Archiv (50 % FIFO-Ringpuffer & 50 % Highscore-Speicher):

- Begrenzung der Archivdateien je Vogelart zur Schonung von Speicherplatz.
- 50 % Chronologischer FIFO-Ringpuffer: Die neuesten Erkennungen bleiben erhalten.
- 50 % Exklusiver Highscore-Speicher: Speichert dauerhaft nur die akustischen Aufnahmen mit dem hoechsten Score (Kombination aus Konfidenz und SNR).

8.4 Externe Schnittstellen & Benachrichtigungen:

- Pushover API: Push-Alerts aufs Smartphone (unterstuetzt Wildcards wie "-Specht", "Alle", "Neu" und individuelle Cooldown-Zeiten gegen Benachrichtigungsspam).
- BirdWeather API: Automatische Weiterleitung validierter Rufe an die weltweite Bioakustik-Forschungsplattform BirdWeather via Queue-Verzeichnis.

- Autostart-Monitor ('autostart_monitor.py'):
Tkinter-basierter Watchdog mit Prozesspruefung alle 60 Sekunden, Mehrfach-Instanzen-Bereinigung und automatischem Konsolen-Neustart.

9. ENTWICKLUNGSPARADIGMA: GEMINI AGENT-BASED VIBECODING

Dieses Projekt entstand in einem modernen, agilen "Agent-based Vibecoding"-Prozess unter Einsatz von Google DeepMind Gemini-Modellen in Antigravity:

- Mensch-KI-Symbiose:
Der Entwickler steuerte Vision, biologisches Fachwissen, Zielarchitektur und praktische Tests mit echten Audioaufnahmen bei. Der Gemini-Coding-Agent uebernahm die Codegenerierung, Signalverarbeitungs-Mathematik, Refactorings, CSS/JS-UI-Gestaltung und API-Integrationen in schnellen, iterativen Feedback-Schleifen.
- Cross-Machine Entwicklung:
Die Entwicklung erfolgte parallel auf zwei verschiedenen Arbeitsstationen (Entwicklungs-PC und Feld-/Bioakustik-Station). Waehrend lokale Transkripte und Caches maschinengebunden sind, bildet das zentrale Git-Repository auf GitHub die vollstaendige, zusammenhaengende Entwicklungsgeschichte ab.

10. PROJEKTSTATISTIKEN & METRIKEN

10.1 Entwicklungszeitraum & Meilensteine:

- Erster Git-Commit: 17. Juni 2026 (00:09:32 UTC)
- Aktueller Stand: 12. Oktober 2026
- Gesamtdauer: 116 Tage (ca. 4 Monate kontinuierliche Weiterentwicklung)

10.2 Git-Repository & Commits:

- Repository: https://github.com/mobifu1/birds_ai_sound_classifier
- Gesamtanzahl Commits: 277 Commits
- Monatliche Verteilung der Entwicklungszyklen:
* Juni 2026: 55 Commits (Initiales Klassifizierungssystem, PyAudio)
* Juli 2026: 53 Commits (Web-Dashboard, Radar, SNR-Kalkulation)
* August 2026: 88 Commits (Filterkaskaden, Woerterbuch, eBird-Geofence)
* September 2026: 63 Commits (ASL, Phasenanalyse, FFT-Periodizitaet)
* Oktober 2026: 18 Commits (Nachtkacheln, Pushover-Wildcards, V1.3.8-RC2)

10.3 Prompts & AI-Agent-Interaktionen:

- Auf dieser lokalen Maschine registriert:
* 15 Antigravity-Sessions
* 184 User-Prompts (u. a. Einzelsessions mit bis zu 72 Prompts)
- Gesamtschaetzung ueber beide Entwicklungsmaschinen (Cross-Machine Setup):
* Ca. 350 bis 450+ interaktive Vibecoding-Prompts ueber die gesamte Projektlaufzeit.
* Durchschnittlich ca. 1,3 bis 1,6 Prompts pro fertiggestelltem Git-Commit.

10.4 Umfang des Codes (Lines of Code - LoC):

```
- Python-Quellcode:              4.654 Zeilen
    * Hauptanwendung (birds_ai_sound_classifier.py): 4.531 Zeilen
    * Monitor-Watchdog (autostart_monitor.py):         123 Zeilen
- HTML-Templates (14 Web-Views): 5.029 Zeilen
    * settings.html:       1.255 Zeilen
    * index.html:            524 Zeilen
    * weekly.html:           480 Zeilen
    * species.html:          471 Zeilen
    * yearly.html:           385 Zeilen
    * daily.html:            324 Zeilen
    * intersection.html:     282 Zeilen
    * wiki.html:             270 Zeilen
    * delete_entry.html:     227 Zeilen
    * prediction.html:       212 Zeilen
    * territory.html:        190 Zeilen
    * db_edit.html:          156 Zeilen
    * fft.html:              142 Zeilen
    * login/misc:             87 Zeilen
- Cascading Style Sheets (CSS):    420 Zeilen
- Client JavaScript (suncalc):     317 Zeilen
- NETTO-QUELLCODE (Codebase):   10.420 ZEILEN
- BirdNET Klassen-Labels:        6.522 Zeilen (BirdNET 6K Labels)
- Konfigurations- & Textdateien:   852 Zeilen (Dictionary, Docs, Batch)
```

- GESAMTUMFANG PROJEKTDATEIEN: 17.794 ZEILEN

Dokumentation generiert am: 12. Oktober 2026

Autor & Maintainer: Andreas / mobifu1

Pair Programming: Gemini Agent via Google Antigravity